



Ciência da Computação

Estrutura de Dados

Fila

Aldo Moura
aldo.moura@souunit.com.br

Roteiro

- **Filas**
  - Conceito
  - Exemplos de uso
  - Criando a estrutura da fila
  - Inserindo célula
  - Consultando as células
  - Removendo célula
  - Esvaziando a fila
  - Variações de fila

Filas no Mundo Real

account_balance

Fila de Banco

Atendimento sequencial por ordem de chegada.

directions_bus

Ponto de Ônibus

Embarque organizado dos passageiros no veículo.

directions_car

Pedágio

Grupo de carros aguardando sua vez para seguir.

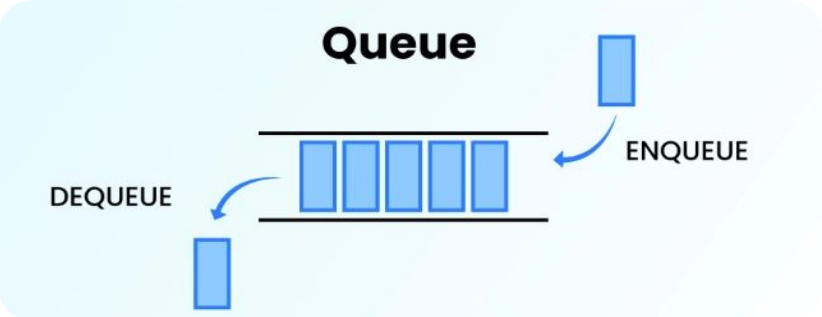
Fila (Queue)

É um caso especial de **lista**, onde a inserção ocorre sempre no **final** e a remoção ocorre sempre no **início**.

FIFO

First In, First Out

Primeiro a Chegar é o Primeiro a Sair



Inserção (Enqueue)

Um novo elemento somente pode ser **inserido** na **última posição** (fim da fila).

Remoção (Dequeue)

Um elemento só pode ser **removido** da **primeira posição** (início da fila).

Exemplos de Uso de Fila

Aplicações práticas de estruturas de dados FILA na computação:

print

Filas de Impressão

Impressoras gerenciam impressões por ordem de chegada; quem envia primeiro é atendido primeiro.

settings_suggest

Filas de Processos

O sistema operacional organiza a execução de múltiplos programas definindo a ordem de prioridade temporal.

assignment

Filas de Tarefas

Conjuntos de dados processados sequencialmente: o dado inserido primeiro será o primeiro a ser atendido.

Criando a Estrutura da Fila

1. Estrutura do Elemento (Nó)

```
typedef struct reg {
   int conteudo;
   struct reg *prox;
} tcelula;
```

Estrutura que representa cada elemento da fila.

- **conteudo:** Armazena o valor do dado.
- ***prox:** Ponteiro para o próximo elemento da fila.

2. Estrutura Auxiliar de Controle das Operações da Fila

```
typedef struct fila {
   tcelula *inicio;
   tcelula *fim;
} tfila;
```

Facilita as operações de inserção e remoção.

- ***inicio:** Aponta para o primeiro elemento (remoção).
- ***fim:** Aponta para o último elemento (inserção).

Inserindo Elemento em uma Fila

```
void insere(int x, tfila *fi)
{
   tcelula *novo = (tcelula*) malloc(sizeof(tcelula));
   novo->conteudo = x;
   novo->prox = NULL;
   if (fi->inicio == NULL) // Caso a fila esteja vazia
   {
      fi->inicio = novo;
      fi->fim = novo;
   }
   else // Caso a fila já tenha elementos
   {
      fi->fim->prox = novo;
      fi->fim = novo;
   }
}
```

Lógica da Inserção:

- **Alocação:** Criação de um novo elemento (tcelula) na memória.
- **Atribuição:** O dado 'x' é armazenado e o ponteiro 'prox' é inicializado com NULL.
- **Encadeamento:** Se a fila estiver vazia, início e fim apontam para o novo nó.
- **Atualização:** Se a fila não estiver vazia, o novo nó entra após o 'fim' atual.

Imprimindo toda a Fila

```
void imprimir(tfila *fi)
{
   if (fi->inicio == NULL)
   {
      printf("\nFila Vazia\n");
   }
   else
   {
      tcelula *aux = fi->inicio;
      printf("\n[");
      do {
         printf(" %d\t", aux->conteudo);
         aux = aux->prox;
      } while (aux != NULL);
      printf("]\n");
   }
}
```

Lógica da Impressão:

- **Verificação:** Checa se o ponteiro de início é nulo para evitar erro em fila vazia.
- **Ponteiro Auxiliar:** Definir um ponteiro 'aux', apontando para o início da fila. O ponteiro será usado para percorrer os demais elementos da fila.
- **Iteração do/while:** Garante que pelo menos um elemento seja impresso e avança para o próximo elemento, até o fim da fila.
- **Formatação:** Imprime colchetes e tabulações para uma visualização clara dos dados.

Remover um Elemento da Fila

```
void remove(tfila *fi)
{
   if (fi->inicio == NULL)
   {
      printf("\nFila Vazia\n");
   }
   else
   {
      tcelula *aux = fi->inicio;
      printf("%d removido", fi->inicio->conteudo);
      fi->inicio = fi->inicio->prox;
      free(aux);
   }
}
```

Lógica da Remoção:

1. Verificação: Confirma se a fila não está vazia antes de tentar remover.

2. Salvaguarda: Usa um ponteiro auxiliar ('aux') para guardar o endereço do primeiro nó.

3. Deslocamento: O 'inicio' da fila passa a apontar para o próximo elemento.

4. Liberação: Libera a memória ocupada pelo nó removido usando 'free'.

Esvaziar a Fila

```
void esvazia(tfila *fi)
{
   if (fi->inicio == NULL)
   {
      printf("\nFila Vazia\n");
   }
   else
   {
      tcelula *aux = fi->inicio;
      do {
         fi->inicio = fi->inicio->prox;
         free(aux);
         aux = fi->inicio;
      } while (aux != NULL);
      printf("\nFila Esvaziada\n");
   }
}
```

Lógica do Esvaziamento:

1. Verificação: Checa se a fila está vazia antes de iniciar o processo.

2. Ponteiro Auxiliar: Definir um ponteiro 'aux', apontando para o início da fila. O ponteiro será usado para percorrer os demais elementos da fila.

3. Loop de Remoção: Percorre a fila enquanto existirem elementos (aux != NULL).

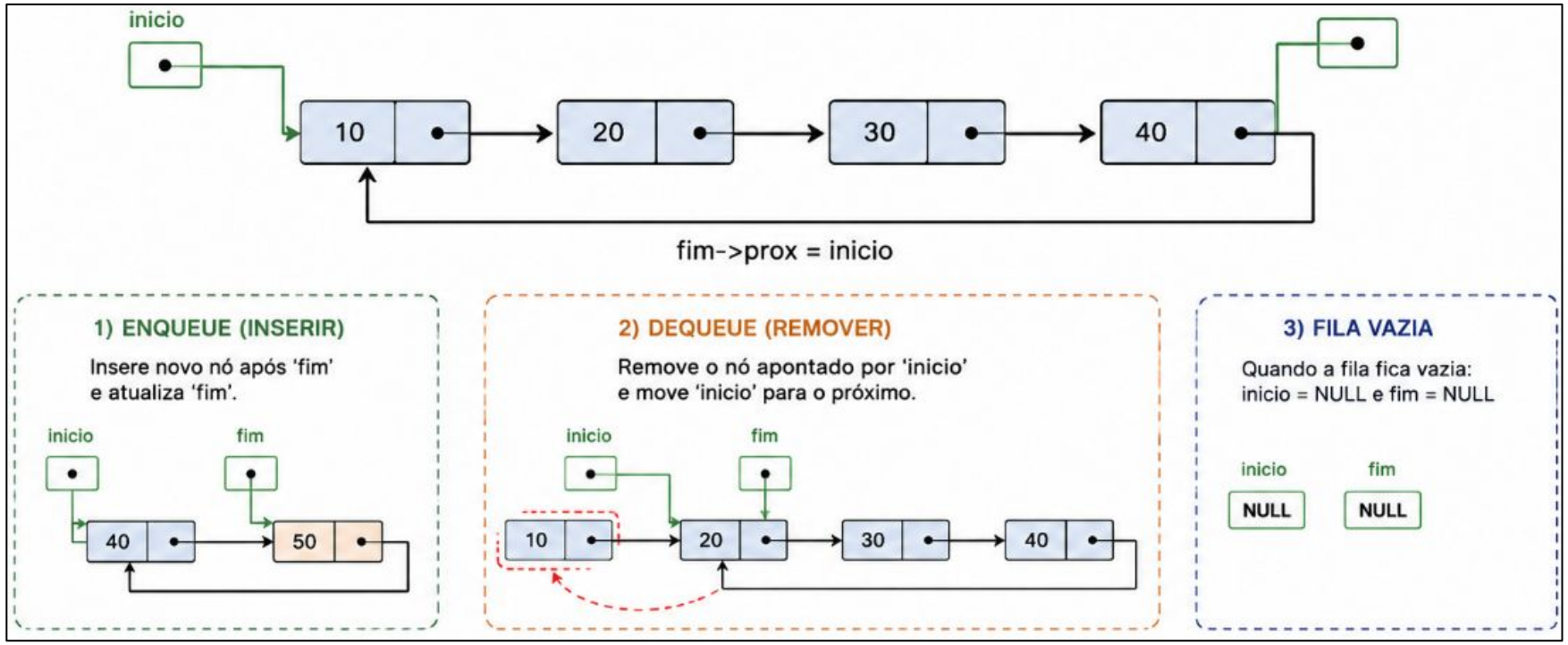
4. Deslocamento e Liberação: Atualiza o início para o próximo nó e libera a memória do nó atual.

5. Finalização: Após remover todos os nós, emite uma mensagem de confirmação.

Variação de Fila: Fila Circular

Segue o princípio **FIFO** (First In, First Out).

O **último elemento aponta para o primeiro**, fechando o ciclo.



Aplicações comuns: Escalonamento de processos e Balanceamento de carga.

Variação de Fila: Deque

Double Ended Queue - Permite inserção e remoção em ambas as extremidades.
Implementado com lista duplamente encadeada.

